APUNTES MAESTROS Y GUÍA DE RESOLUCIÓN: CONTROL CHECK 1 (DP1)

Curso 2025-2026 | Arquitectura Spring Boot, JPA, Rest API & Reactstrap

REGLAS DE ORO DEL EXAMEN (LÍNEAS ROJAS):

1. **SI EL CÓDIGO NO COMPILA O LA APLICACIÓN NO ARRANCA (CONTEXTO SPRING FALLA): LA NOTA ES 0.** Comprueba la sintaxis de tu SQL e imports antes de entregar.
2. **NUNCA modifiques las firmas de los métodos ni los nombres de las clases dadas.** Los tests automáticos de corrección usan reflexión sobre esos nombres exactos.
3. **NUNCA modifiques los tests unitarios.** Serán restaurados vía Git antes de corregir.
4. **Entregas exclusivas mediante `git push`:** Debes hacer push a tu repositorio personal antes de abandonar el aula.

ÍNDICE ESTRUCTURADO POR TIPOS DE EJERCICIOS

Sección	Tema / Tipo de Ejercicio	Página
1. Entidades y Validaciones JPA	Mapeo relacional, diccionario de validaciones (@NotBlank, @Size, @NotNull), Enums y Atributos Derivados.	Pág. 2
2. Relaciones UML a JPA	@ManyToOne, @OneToMany, @ManyToMany, @OneToOne, cardinalidades y gestión de `@JoinColumn`.	Pág. 3
3. Script de Base de Datos (data.sql)	Orden de inserción, sintaxis H2, fechas, valores nulos y mapeo manual de tablas intermedias.	Pág. 4
4. Servicios y Reglas de Negocio	Anotaciones @Transactional, excepciones personalizadas y prevención de NullPointerExceptions (Error 500).	Pág. 5
5. Repositorios y Consultas (JPQL)	Derived Queries mágicas, consultas compuestas, JOINs, agregados (SUM CASE) y native queries.	Pág. 6
6. Controladores REST y Seguridad	Mapeo de endpoints, respuestas ResponseEntity, manejo de 404 NOT_FOUND y 403 FORBIDDEN sin fallos.	Pág. 7
7. Frontend React & Reactstrap	Hooks (useState, useEffect), manejo de APIs con fetch, tablas dinámicas y validación con Jest.	Pág. 8
8. Matriz de Decisión y Debugging	Tabla de resolución rápida ante enunciados nuevos y solución a errores de consola comunes.	Pág. 9

FLUIJO DE TRABAJO Y ENTORNO (COMANDOS CLAVE)

- **Ejecutar backend con tests:** `./mvnw test` (en la raíz del proyecto).
- **Ejecutar backend omitiendo tests:** `./mvnw spring-boot:run`
- **Ejecutar frontend:** Entrar en `frontend/`, ejecutar `npm install` (solo 1ª vez) y `npm test` (pulsar 'a' para lanzar todos los tests).
- **Liberar Puerto 8080 bloqueado (Windows):** `netstat -ano | findstr :8080` y luego `taskkill /PID [PID] /F`.

1. ENTIDADES Y MAPEO BÁSICO (JPA)

1.1. Estructura de Entidad Básica

Si la entidad hereda de `BaseEntity`, **no debes declarar el atributo ID**, ya viene heredado.

```
@Entity
@Table(name = "custom_table_name") // Solo si el enunciado pide un nombre de tabla explícito
@Getter @Setter
public class Tournament extends BaseEntity {
    // No poner 'private Integer id;' si extienda de BaseEntity.
    // Si NO hereda de BaseEntity:
    // @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Integer id;
}
```

1.2. Diccionario de Validaciones (Jakarta / Bean Validation)

Tipo de Dato	Anotación	Uso / Regla de negocio
String	`@NotBlank`	Obligatorio. Falla si es null, si está vacío `""` o solo tiene espacios.
String	`@Size(min=X, max=Y)`	Establece longitud mínima y máxima del texto (ej. `min=3, max=50`).
String	`@Email` / `@URL`	Valida formato de correo o dirección web.
String	`@Pattern(regexp="...")`	Expresiones regulares (DNI, códigos con formato fijo).
Números	`@NotNull`	Obligatorio. (No usar @NotBlank en números o fechas).
Números	`@Min(X)` / `@Max(Y)`	Límites numéricos. Ej: `@Min(2)` para maxParticipants, `@Min(0)` para victorias.
Fechas	`@NotNull`	Obligatorio en LocalDate o LocalDateTime.
Fechas	`@Past` / `@Future`	Exige que la fecha sea estrictamente anterior o posterior al momento actual.

1.3. Enumerados (@Enumerated)

> **OBLIGATORIO:** Siempre se debe especificar `EnumType.STRING` para guardar el texto del enum en la BD en lugar de su ordinal entero.

```
@NotNull
@Enumerated(EnumType.STRING)
private AnnouncementStatus status; // Guarda "DRAFT", "SENT", "ARCHIVED" en la BD
```

Importante: Recuerda incluir arriba `import jakarta.persistence.EnumType;` para evitar el fallo de compilación `EnumType cannot be resolved`.

1.4. Atributos Calculados (El truco del @Transient)

Si el diagrama UML indica un atributo antecedido por una barra `/winRate`, significa que es derivado y **no debe persistirse en la base de datos**.

```
@Transient // JPA e Hibernate ignoran este campo en las tablas SQL
public Double getWinRate() {
    int total = (wins != null ? wins : 0) + (losses != null ? losses : 0);
    if (total == 0) return 0.0;
    return (double) wins / total;
}
```

2. RELACIONES UML A ANOTACIONES JPA

Para interpretar una relación del diagrama UML, analiza la cardinalidad del extremo navegable (a donde apunta la flecha o la asociación):

- Si el extremo navegable es `0..1` o `1` $\rightarrow$ Usar `@ManyToOne` o `@OneToOne`.
- Si el extremo navegable es `0..n` o `1..n` $\rightarrow$ Usar `@OneToMany` o `@ManyToMany`.
- **Obligatoriedad:** Si la cardinalidad mínima en el extremo destino es `1` o `1..n`, añade `@NotNull` a la relación.

2.1. @ManyToOne (La más común)

Usada cuando el objeto hijo contiene la clave foránea del padre.

```
// En la clase Move o Rating hacia User/ChessMatch
@NotNull // Si el UML marca cardinalidad 1 (obligatorio)
@ManyToOne(optional = false)
@JoinColumn(name = "match_id") // Clave foránea explícita
private ChessMatch match;
```

2.2. @OneToMany (Unidireccional y Bidireccional)

Unidireccional (El padre conoce a la lista de hijos): Debe incluir `@JoinColumn` para evitar la creación de una tabla intermedia innecesaria salvo que sea explícita.

```
// Relación Unidireccional desde Tournament hacia ChessMatch (matches: 0..n)
@OneToMany
@JoinColumn(name = "tournament_id")
private List<ChessMatch> matches;
```

Bidireccional (Ambas entidades se conocen): Se usa `mappedBy` en la clase padre para señalar que la propiedad ya la gestiona el `@ManyToOne` del hijo.

```
// En la clase Padre (ej. Community)
@OneToMany(mappedBy = "community", cascade = CascadeType.ALL)
private List<Season> seasons;
```

2.3. @ManyToMany (Muchos a Muchos)

```
// Relación desde Community hacia User (members)
@ManyToMany
@JoinTable(
    name = "community_members",
    joinColumns = @JoinColumn(name = "community_id"),
    inverseJoinColumns = @JoinColumn(name = "user_id")
)
private List<User> members;
```

> **ATENCIÓN AL QUITAR @TRANSIENT:** Al implementar un ejercicio de examen de tipo "Modificar entidades para mapear relaciones", debes **eliminar la anotación `@Transient` ** de los atributos correspondientes que venían predefinidos en la plantilla inicial.

3. SCRIPT DE BASE DE DATOS (data.sql)

Si la aplicación falla al arrancar por un error SQL, obtendrás un 0 en el examen. Sigue este procedimiento estricto para modificar el archivo `data.sql` o `import.sql`:

3.1. Orden Estricto de Inserción

1. **Entidades Independientes (Sin Foreign Keys):** Insertar primero `Users`, `ChessBoard`, `ProblemSource`.
2. **Entidades Dependientes:** Insertar `ChessMatch`, `Tournament`, `Move`, `Rating` (hacen referencia a los IDs anteriores).
3. **Tablas Intermedias de Asociación:** Insertar las relaciones `@ManyToMany` o `@OneToMany` unidireccionales (ej. `season_ratings`, `tournament_matches`).

3.2. Formato de Datos en H2 SQL

- **Fechas (LocalDate):** Formato estricto `'YYYY-MM-DD'` $\rightarrow$ `'2024-09-01'`.
- **Tiempos (LocalDateTime):** Formato estricto `'YYYY-MM-DD HH:MM:SS'` $\rightarrow$ `'2024-09-01 10:00:00'`.
- **Comillas:** Usar comillas simples `'texto'`. Para incluir un apóstrofe en el texto, usa doble comilla simple: `'I''m ready'`.
- **Comas y Puntos y Comas:** Separa cada tupla de valores con comas `,` y cierra el statement completo con punto y coma `;`.

3.3. Código de Referencia SQL para data.sql

```
-- 1. Inserción de Torneos (Entidad Principal)
INSERT INTO tournament (id, name, location, start_date, end_date, max_participants) VALUES
(1, 'Abierto de Ajedrez Villa de Los Palacios', 'Circulo Joven', '2024-11-10 09:00:00', '2024-11
(2, 'Torneo en Honor de José Maestre', 'Club El Casino', '1990-09-17 17:00:00', '1990-09-20 21:0

-- 2. Inserción de Movimientos con claves foráneas (match_id, piece_id, capture_id)
INSERT INTO move (id, turn, originx, originy, destinationx, destinationy, match_id, piece_id, ca
(1, 1, 4, 2, 4, 4, 15, 131, null),
(2, 7, 7, 5, 5, 6, 5, 5001, 5002);

-- 3. Inserción en Tablas Intermedias (@OneToMany Unidireccional o @ManyToMany)
INSERT INTO tournament_matches (tournament_id, matches_id) VALUES
(1, 15),
(2, 5),
(2, 6);
```

4. SERVICIOS (@Service) Y REGLAS DE NEGOCIO

Los métodos de los servicios gestionan la lógica de negocio y las transacciones con la base de datos.

- **Anotación `@Transactional`:** Debe colocarse sobre los métodos del servicio. Usa `@Transactional(readOnly = true)` en consultas y `@Transactional(rollbackFor = ExcepcionCustom.class)` en escrituras.
- **Lanzar Excepción de Negocio:** Si se incumple una regla, se debe lanzar la excepción personalizada indicada en el enunciado (ej. `UnfeasibleMatchException`, `MalformedCommunityException`).

4.1. Patrón para Validaciones de Negocio Complejas

```
@Service
public class ChessMatchService {

    @Autowired
    private ChessMatchRepository matchRepository;

    @Transactional(rollbackFor = UnfeasibleMatchException.class)
    public ChessMatch save(ChessMatch match) throws UnfeasibleMatchException {

        // REGLA 1: El ganador debe ser nulo o ser uno de los dos jugadores (whitePlayer o black
        if (match.getWinner() != null) {
            boolean isWhite = match.getWinner().equals(match.getWhitePlayer());
            boolean isBlack = match.getWinner().equals(match.getBlackPlayer());
            if (!isWhite && !isBlack) {
                throw new UnfeasibleMatchException("El ganador debe ser uno de los participantes
            }
        }

        // REGLA 2: El instante de inicio debe ser anterior al fin (si el fin no es nulo)
        if (match.getStart() != null && match.getFinish() != null) {
            if (!match.getStart().isBefore(match.getFinish())) {
                throw new UnfeasibleMatchException("La fecha de inicio debe ser anterior al fin.
            }
        }

        return matchRepository.save(match);
    }
}
```

> **Prevención de NullPointerExceptions (Evitar Error 500 en Tests):**
> Cuando iteres colecciones en un servicio (ej. comprobar partidas de miembros de una comunidad), valida siempre que la lista y sus elementos no sean nulos: `if (comunidad.getMembers() != null) { for (User u : comunidad.getMembers()) { ... } }`.

5. REPOSITORIOS Y CONSULTAS PERSONALIZADAS (JPQL)

5.1. Derived Queries (Consultas por Nombre de Método)

Spring Data JPA genera el código automáticamente según el nombre del método en la interfaz:

```
@Repository
public interface PieceRepository extends CrudRepository<Piece, Integer> {
    // Busca piezas filtrando por su atributo 'color' y por 'name' de su 'PieceType'
    List<Piece> findByColorAndPieceTypeName(PieceColor color, String name);
}
```

5.2. Consultas Complejas con @Query (JPQL) y Agregados (SUM / CASE)

Si la consulta requiere lógica condicional (como asignar 2 puntos al ganador, 1 por tablas y 0 al perdedor), utiliza un bloque JPQL con `SUM(CASE WHEN ...)`:

```
@Repository
public interface FriendlyChallengeRepository extends CrudRepository<FriendlyChallenge, Integer>

    @Query("SELECT u, " +
           "SUM(CASE " +
           "  WHEN m.winner = u THEN 2 " +
           "  WHEN m.winner IS NULL AND (m.whitePlayer = u OR m.blackPlayer = u) THEN 1 " +
           "  ELSE 0 " +
           "END) " +
           "FROM FriendlyChallenge fc " +
           "JOIN fc.participants u " +
           "JOIN fc.matches m " +
           "WHERE fc.id = :challengeId " +
           "  AND m.finish IS NOT NULL " +
           "  AND (m.whitePlayer = u OR m.blackPlayer = u) " +
           "GROUP BY u " +
           "ORDER BY 2 DESC")
    List<Object[]> findParticipantPoints(@Param("challengeId") Integer challengeId);
}
```

Nota de Ordenación: JPQL permite usar `ORDER BY 2 DESC` para ordenar por la segunda columna seleccionada en la cláusula `SELECT` (en este caso, la suma total de puntos).

6. CONTROLADORES REST Y CONTROL DE SEGURIDAD

El controlador expone la API REST del sistema. Debe manejar adecuadamente los códigos HTTP de respuesta:

- `200 OK`: Lectura u orden de procesamiento exitosa.
- `404 NOT_FOUND`: El recurso solicitado por ID no existe en la BD.
- `403 FORBIDDEN`: El usuario autenticado no tiene la autoridad exigida (ej. `PLAYER`).

6.1. Patrón Infalible de Seguridad Manual para Evitar 500/404 en Tests

Para evitar que los tests fallen con `Expected 403 but got 404/500` al consultar usuarios en base de datos que no existen en H2, evalúa el contexto de seguridad en memoria mediante `SecurityContextHolder`:

```
@RestController
@RequestMapping("/api/v1/attempts")
public class ChessProblemAttemptController {

    @Autowired
    private ChessProblemAttemptService attemptService;

    // Método auxiliar de comprobación de autoridades
    private void checkPlayerRole() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        boolean isPlayer = auth != null && auth.getAuthorities().stream()
                .anyMatch(a -> "PLAYER".equals(a.getAuthority()));

        if (!isPlayer) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Acceso Denegado");
        }
    }

    @GetMapping
    public ResponseEntity<List<ChessProblemAttempt>> getAll() {
        checkPlayerRole();
        return ResponseEntity.ok(attemptService.getAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<ChessProblemAttempt> getById(@PathVariable("id") Integer id) {
        checkPlayerRole();
        ChessProblemAttempt attempt = attemptService.getById(id);
        if (attempt == null) {
            throw new ResourceNotFoundException("Attempt", "id", id);
        }
        return ResponseEntity.ok(attempt);
    }
}
```

7. FRONTEND REACT (Reactstrap & Jest Tests)

Los componentes React del examen (ubicados en `frontend/src/.../index.js`) se evalúan con Jest. Debes respetar los títulos exactos de las columnas y los textos condicionales.

7.1. Plantilla Universal de Componente de Listado

```
import React, { useEffect, useState } from "react";
import { Table } from "reactstrap";

export default function Standings({ seasonId }) {
    const [ratings, setRatings] = useState([]);

    useEffect(() => {
        // Petición GET a la API con parámetro en la URL
        fetch(`/api/v1/seasons/${seasonId}/standings`)
            .then((r) => r.ok ? r.json() : [])
            .then((data) => setRatings(Array.isArray(data) ? data : []))
            .catch(() => setRatings([]));
    }, [seasonId]);

    // 1. CONDICIONAL DE ARRAY VACÍO (Requisito indispensable de examen)
    if (!ratings || ratings.length === 0) {
        return <h1>NOT ENOUGH MATCHES</h1>; // Texto exacto pedido en el enunciado
    }

    // 2. RENDERIZADO DE TABLA
    return (
        <Table role="table" striped>
            <thead>
                <tr>
                    <th>Player</th>
                    <th>ELO</th>
                    <th>Wins</th>
                    <th>Losses</th>
                    <th>Draws</th>
                    <th>Win %</th>
                </tr>
            </thead>
            <tbody>
                {ratings.map((r) => (
                    <tr key={r.id}>
                        <td>{r.player?.username ?? "-"}</td>
                        <td>{r.elo}</td>
                        <td>{r.wins}</td>
                        <td>{r.losses}</td>
                        <td>{r.draws}</td>
                        <td>{r.winRate ? (r.winRate * 100).toFixed(2) + "%" : "0%"}</td>
                    </tr>
                ))}
            </tbody>
        </Table>
    );
}
```

8. GUÍA DE DECISIÓN Y COMPROBACIONES DE EXAMEN

Si el enunciado pide...	Debo revisar / modificar...	Qué debo implementar...	Comprobación / Test
Guardar entidad con reglas de negocio	`Service` y clase de Excepción personalizada.	Añadir validaciones en el método `save()` y lanzar la excepción con `@Transactional`.	`./mvnw test` (Test 1)
Crear servicio y endpoints de API REST	`Service`, `Repository`, `Controller`.	Métodos CRUD, derivated queries o JPQL, retornos `ResponseEntity` y check de seguridad 403.	`./mvnw test` (Test 2 / 4)
Crear vista de listado en React	`frontend/src/.../index.js`	Petición `fetch` en `useEffect`, renderizado condicional de `<h1>TEXTO</h1>` y `Table`.	`npm test` (en carpeta frontend)
Crear nuevas entidades y mapear UML	Clases del modelo backend y repositorios `CrudRepository`.	Añadir `@Entity`, `@NotBlank`, `@Size`, `@Enumerated`, relaciones (`@ManyToOne`, etc.) y quitar `@Transient`.	Compilación en IDE
Cargar datos iniciales en el sistema	`src/main/resources/data.sql`	Sentencias `INSERT INTO` respetando el orden de FKs y tablas intermedias.	Arrancar app sin errores de contexto.

LISTA DE COMPROBACIÓN FINAL ANTES DE ENTREGAR (GIT PUSH)

- [ ] **Compilación Java:** Ejecuta `./mvnw test-compile` y verifica que no hay ningún error sintáctico.
- [ ] **Imports correctos:** Asegúrate de que `EnumType` proviene de `jakarta.persistence.EnumType`.
- [ ] **Tests pasados:** Revisa el porcentaje de tests aprobados con `./mvnw test`.
- [ ] **Frontend sin fallos:** Revisa `npm test` dentro de la carpeta `frontend`.
- [ ] **Git Push final:** Ejecuta `git status`, `git add .`, `git commit -m "Solucion final"` y `git push`.